

Branch Protection Recommendations

Overview

Branch protection rules help maintain code quality and prevent accidental changes to important branches. This document provides recommended settings for the BDI project.

Protected Branches

Main Branch (`main`)

The `main` branch should have the strictest protection rules as it represents production-ready code.

Recommended Settings

Require Pull Request Reviews:

- ☒ Require pull request reviews before merging
- Number of required approvals: **2**
- ☒ Dismiss stale pull request approvals when new commits are pushed
- ☒ Require review from Code Owners (if CODEOWNERS file exists)
- ☒ Require approval of the most recent reviewable push

Require Status Checks:

- ☒ Require status checks to pass before merging
- ☒ Require branches to be up to date before merging

Required Status Checks:

- `build-and-test (gcc-13, debug)`
- `build-and-test (gcc-13, release)`
- `build-and-test (clang-16, debug)`
- `build-and-test (clang-16, release)`
- `sanitizer-checks (address)`
- `sanitizer-checks (undefined)`
- `format-check`

Require Conversation Resolution:

- ☒ Require conversation resolution before merging

Require Signed Commits:

- ☒ Require signed commits (recommended for security)

Require Linear History:

- ☒ Require linear history (prevents merge commits)
- Alternative: Allow merge commits but require squash

Include Administrators:

- ☒ Include administrators (enforce rules for everyone)

Restrict Pushes:

- ☒ Restrict who can push to matching branches
- Allowed: Repository administrators only

Allow Force Pushes:

- ❌ Do not allow force pushes

Allow Deletions:

- ❌ Do not allow branch deletion

Development Branch (`develop`)

The `develop` branch serves as an integration branch for features.

Recommended Settings**Require Pull Request Reviews:**

- ✅ Require pull request reviews before merging
- Number of required approvals: **1**
- ✅ Dismiss stale pull request approvals when new commits are pushed

Require Status Checks:

- ✅ Require status checks to pass before merging
- ⚠️ Do not require branches to be up to date (allows faster integration)

Required Status Checks:

- `build-and-test (gcc-13, debug)`
- `build-and-test (clang-16, debug)`
- `sanitizer-checks (address)`

Require Conversation Resolution:

- ✅ Require conversation resolution before merging

Include Administrators:

- ⚠️ Optional: May exclude administrators for faster iteration

Allow Force Pushes:

- ❌ Do not allow force pushes

Allow Deletions:

- ❌ Do not allow branch deletion

Release Branches (`release/*`)

Release branches prepare code for production releases.

Recommended Settings**Require Pull Request Reviews:**

- ✅ Require pull request reviews before merging
- Number of required approvals: **2**
- ✅ Require review from Code Owners

Require Status Checks:

- ✅ Require status checks to pass before merging
- ✅ Require branches to be up to date before merging

Required Status Checks:

- All CI checks (same as `main`)
- Additional: Performance benchmarks
- Additional: Security scans

Require Signed Commits:

- ☒ Require signed commits

Allow Force Pushes:

- ☒ Do not allow force pushes

Allow Deletions:

- ☒ Allow deletion after merge to `main`

Hotfix Branches (`hotfix/*`)

Hotfix branches address critical production issues.

Recommended Settings**Require Pull Request Reviews:**

- ☒ Require pull request reviews before merging
- Number of required approvals: **1** (expedited for urgency)
- ☒ Require review from Code Owners

Require Status Checks:

- ☒ Require status checks to pass before merging
- ☒ Require branches to be up to date before merging

Required Status Checks:

- `build-and-test (gcc-13, release)`
- `build-and-test (clang-16, release)`
- `sanitizer-checks (address)`

Allow Force Pushes:

- ☒ Do not allow force pushes

Allow Deletions:

- ☒ Allow deletion after merge

GitHub Settings Configuration

Via GitHub Web Interface

1. Navigate to repository settings
2. Click “Branches” in the left sidebar
3. Click “Add rule” under “Branch protection rules”
4. Enter branch name pattern (e.g., `main`)
5. Configure protection settings as recommended above
6. Click “Create” or “Save changes”

Via GitHub CLI

```
# Install GitHub CLI
# https://cli.github.com/

# Protect main branch
gh api repos/Mecca-Research/The-Binary-Decomposition-Interface/branches/main/protection \
  --method PUT \
  --field required_status_checks='{ "strict":true,"contexts":["build-and-test (gcc-13, debug)", "build-and-test (gcc-13, release)", "build-and-test (clang-16, debug)", "build-and-test (clang-16, release)", "sanitizer-checks (address)", "sanitizer-checks (undefined)", "format-check"] }' \
  --field enforce_admins=true \
  --field required_pull_request_reviews='{ "dismissal_restrictions":{ }, "dismiss_stale_reviews":true, "require_code_owner_reviews":true, "required_approving_review_count":2 }' \
  --field restrictions=null \
  --field required_linear_history=true \
  --field allow_force_pushes=false \
  --field allow_deletions=false
```

CODEOWNERS File

Create a `.github/CODEOWNERS` file to automatically request reviews from specific people or teams:

```
# BDI Code Owners

# Default owners for everything
* @Mecca-Research/core-team

# Compiler toolchain
/C/compiler/ @Mecca-Research/compiler-team

# Kernel layer
/C/kernel/ @Mecca-Research/kernel-team

# VM and execution
/C/vm/ @Mecca-Research/vm-team

# Documentation
/docs/ @Mecca-Research/docs-team

# CI/CD
/.github/ @Mecca-Research/devops-team

# Build system
/Makefile @Mecca-Research/build-team
/build_config.mk @Mecca-Research/build-team
```

Pull Request Template

Create `.github/PULL_REQUEST_TEMPLATE.md` :

Description

<!-- Provide a brief description of the changes -->

Type of Change

- [] Bug fix (non-breaking change which fixes an issue)
- [] New feature (non-breaking change which adds functionality)
- [] Breaking change (fix or feature that would cause existing functionality to not work as expected)
- [] Documentation update
- [] Performance improvement
- [] Code refactoring
- [] Test addition/improvement

Related Issues

<!-- Link to related issues: Fixes #123, Closes #456 -->

Changes Made

<!-- List the main changes made in this PR -->

-
-
-

Testing

<!-- Describe the tests you ran to verify your changes -->

- [] Unit tests pass (`make test`)
- [] Integration tests pass
- [] Manual testing performed
- [] Tested with sanitizers (ASan, UBSan)
- [] Performance benchmarks run (if applicable)

Checklist

- [] My code follows the project's style guidelines
- [] I have performed a self-review of my code
- [] I have commented my code, particularly in hard-to-understand areas
- [] I have made corresponding changes to the documentation
- [] My changes generate no new warnings
- [] I have added tests that prove my fix is effective or that my feature works
- [] New and existing unit tests pass locally with my changes
- [] Any dependent changes have been merged and published

Screenshots (if applicable)

<!-- Add screenshots to help explain your changes -->

Additional Notes

<!-- Any additional information that reviewers should know -->

Issue Templates

Bug Report Template

Create `.github/ISSUE_TEMPLATE/bug_report.md` :

```

---
name: Bug Report
about: Create a report to help us improve
title: '[BUG] '
labels: bug
assignees: ''
---

## Bug Description

<!-- A clear and concise description of what the bug is -->

## To Reproduce

Steps to reproduce the behavior:
1.
2.
3.

## Expected Behavior

<!-- A clear and concise description of what you expected to happen -->

## Actual Behavior

<!-- What actually happened -->

## Environment

- OS: [e.g., Ubuntu 22.04]
- Compiler: [e.g., GCC 13.1]
- Build Mode: [e.g., debug, release]
- Commit/Branch: [e.g., main@abc123]

## Additional Context

<!-- Add any other context about the problem here -->

## Possible Solution

<!-- Optional: Suggest a fix or reason for the bug -->

```

Feature Request Template

Create `.github/ISSUE_TEMPLATE/feature_request.md` :

```

---
name: Feature Request
about: Suggest an idea for this project
title: '[FEATURE] '
labels: enhancement
assignees: ''
---

## Feature Description

<!-- A clear and concise description of the feature -->

## Motivation

<!-- Why is this feature needed? What problem does it solve? -->

## Proposed Solution

<!-- Describe how you envision this feature working -->

## Alternatives Considered

<!-- Describe alternative solutions or features you've considered -->

## Additional Context

<!-- Add any other context or screenshots about the feature request -->

## Implementation Notes

<!-- Optional: Technical details about implementation -->

```

Enforcement Strategy

Automated Enforcement

1. **CI/CD Pipeline:** All checks must pass before merge
2. **Pre-commit Hooks:** Local validation before commit
3. **Branch Protection:** GitHub enforces rules automatically

Manual Review Process

1. **Code Review Checklist:**
 - ☐ Code follows style guidelines
 - ☐ Changes are well-documented
 - ☐ Tests are comprehensive
 - ☐ No security vulnerabilities introduced
 - ☐ Performance impact is acceptable
 - ☐ Breaking changes are documented
2. **Review Timeline:**
 - Initial review: Within 24 hours
 - Follow-up reviews: Within 48 hours
 - Urgent hotfixes: Within 4 hours
3. **Approval Requirements:**
 - **main:** 2 approvals from core team

- **develop**: 1 approval from any team member
- **feature branches**: 1 approval recommended

Exceptions and Overrides

When to Override Protection

- **Critical Hotfixes**: Security vulnerabilities requiring immediate fix
- **Build System Issues**: CI/CD pipeline failures preventing merges
- **Documentation Updates**: Low-risk documentation-only changes

Override Process

1. Document reason for override in PR description
2. Get approval from repository administrator
3. Merge with administrator privileges
4. Create follow-up issue to address skipped checks
5. Log override in project changelog

Override Logging

Maintain a log of protection overrides:

```
Date: 2024-01-15
Branch: main
PR: #456
Reason: Critical security vulnerability (CVE-2024-XXXX)
Approved by: @admin
Skipped checks: None (emergency merge)
Follow-up: Issue #457 created for post-merge validation
```

Monitoring and Maintenance

Regular Reviews

- **Monthly**: Review branch protection effectiveness
- **Quarterly**: Update required status checks
- **Annually**: Comprehensive policy review

Metrics to Track

- Pull request merge time
- Number of protection overrides
- CI/CD failure rates
- Code review participation

Continuous Improvement

- Gather feedback from team members
- Adjust rules based on project needs
- Balance security with development velocity
- Document lessons learned

References

- [GitHub Branch Protection Documentation](https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches) (https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches)
- [GitHub CODEOWNERS Documentation](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners) (https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners)
- [Conventional Commits](https://www.conventionalcommits.org/) (https://www.conventionalcommits.org/)